

# Point Cloud Part Editing: Segmentation, Generation, Assembly, and Selection

Kaiyi Zhang<sup>1</sup>, Yang Chen<sup>1</sup>, Ximing Yang<sup>1</sup>, Weizhong Zhang<sup>1,2</sup>, Cheng Jin<sup>1,2</sup>

<sup>1</sup>School of Computer Science, Fudan University, Shanghai, China

<sup>2</sup>Innovation Center of Calligraphy and Painting Creation Technology, MCT, China  
{zhangky20, chen\_yang19, xmyang19, weizhongzhang, jc}@fudan.edu.cn

## Abstract

Ideal part editing should guarantee the diversity of edited parts, the fidelity to the remaining parts, and the quality of the results. However, previous methods do not disentangle each part completely, which means the edited parts will affect the others, resulting in poor diversity and fidelity. In addition, some methods lack constraints between parts, which need manual selections of edited results to ensure quality. Therefore, we propose a four-stage process for point cloud part editing: Segmentation, Generation, Assembly, and Selection. Based on this process, we introduce **SGAS**, a model for part editing that employs two strategies: feature disentanglement and constraint. By independently fitting part-level feature distributions, we realize the feature disentanglement. By explicitly modeling the transformation from object-level distribution to part-level distributions, we realize the feature constraint. Considerable experiments on different datasets demonstrate the efficiency and effectiveness of SGAS on point cloud part editing. In addition, SGAS can be pruned to realize unsupervised part-aware point cloud generation and achieves state-of-the-art results.

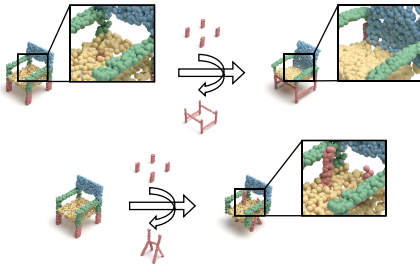
## Introduction

In the context of 3D object modeling, parts are considered the fundamental units. Recently, part-based methods (?????) have become more and more prevailing. These methods typically involve obtaining different parts first and then assembling them. Although many works have explored procedural content generation (?), which is often used to make material maps and game maps, the rapid development of game scenes still relies heavily on the generation of 3D objects. Part-based methods enable part editing, which involves replacing some parts of an object to create a new one, thereby enhancing the diversity of 3D modeling.

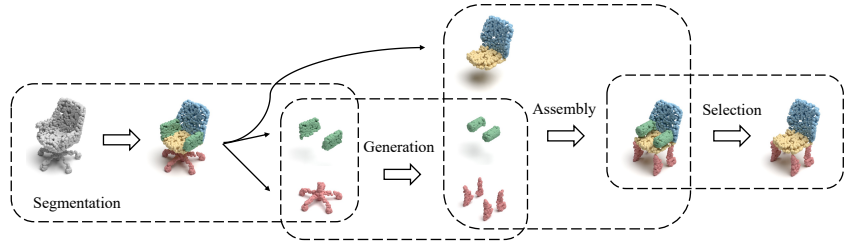
Ideal part editing should make edited parts diverse while keeping unedited parts unchanged to form a reasonable object. These correspond to three important properties of the edited results: diversity, fidelity, and quality. However, when previous part-based methods are applied to part editing, they have two problems. As shown in Figure 1(a), on the one hand, methods such as MRGAN (?) and SP-GAN (?) do not realize radical disentanglement between parts. Therefore, when some parts are modified, other parts will also

change, which means they do not guarantee the fidelity to the remaining parts. Similarly, multimodal shape completion methods such as MSC-cGAN (?), which can be regarded as a subset of part editing, also do not disentangle parts. This not only changes the input parts but also makes the completion results less diverse. Someone may argue that the adjacent parts may change to accommodate the edited parts, but this will not affect the requirement of radical disentanglement, since all changed parts can be regarded as edited parts. On the other hand, some methods (???) do not implement constraints between parts effectively. This may result in poor part assembly and mismatched parts to be used in the formation of an object. Although these methods attempt to achieve assembly by moving parts, the changed parts still need a manual selection to ensure high-quality edited results that lead to a reasonable object.

To address these issues, as shown in Figure 1(b), we first propose a four-stage process for point cloud part editing: segmentation, generation, assembly, and selection. For the three properties of edited results, segmentation can guarantee the fidelity to the remaining parts by isolating the parts; generation can guarantee the diversity of edited parts by exploring different variations of the parts; assembly and selection can guarantee the quality of the results by choosing the most appropriate edited parts and assembling them in a coherent manner. Based on this process, we introduce a model SGAS for part editing that employs two strategies: feature disentanglement and constraint. We first use unsupervised shape co-segmentation methods (???) or manual segmentation to obtain Ground Truth parts. Then we pre-train several autoencoders at the part level. Finally, by adversarially supervising part-level feature transformations, we realize the feature disentanglement during generation. Since each part is generated separately, this not only ensures the diversity of edited parts but also the fidelity to the remaining parts. In addition, we make the distribution of each part transformed from the same Gaussian distribution and adversarially supervise the generations of all parts simultaneously to ensure that edited parts can assemble well and form a reasonable object. This strategy is called feature constraint. It guarantees the quality of the edited results. By adding a part selection module to the final output part features, which allows SGAS to autonomously choose which parts do not need to be output, the quality of the edited results can be further improved.



(a) Two problems with existing part editing methods



(b) A four-stage point cloud part editing process

Figure 1: (a) Top: w/o disentanglement between parts. When changing the chair base, the other parts such as the chair back, arm, and seat will also change. Bottom: w/o constraints between parts. The changed chair base is not only poorly assembled but also does not match other parts, resulting in the generation of an unreasonable object. (b) It includes four stages: segmentation, generation, assembly, and selection, which can guarantee fidelity, diversity, and quality of the edited results respectively.

Our main contributions are the following:

- We propose a novel point cloud part editing process. It includes four stages: segmentation, generation, assembly, and selection.
- Based on the proposed process, we introduce SGAS, a model for part editing that employs two strategies of feature disentanglement and constraint. Experiments show that SGAS achieves excellent quantitative and qualitative part editing results.
- A new diversity metric of edited results: Total Mutual Difference Surface (TMDS).
- SGAS can be pruned to realize unsupervised part-aware point cloud generation and achieves state-of-the-art results on the ShapeNet-Partseg dataset.

## Related Work

### Part-based Shape Generation

**Unsupervised Part-aware Point Cloud Generation** The fine-grained improvement of generative results can be achieved through local generation. Therefore, many methods attempt to explore the generation of multiple parts and combine them into a final shape. Since part-level ground truth data is often unavailable, these methods typically involve unsupervised segmentation of parts. For example, TreeGAN (?) first designs the generation of the point cloud as a tree growth process and then combines the various parts at the leaf nodes. To achieve controllable point cloud generation, SP-GAN (?) is proposed. Similar to FoldingNet (?), SP-GAN transforms a sphere in 3D space into a target point cloud, where different parts of the 3D sphere correspond to different parts of the target point cloud. MRGAN (?) explicitly realizes part disentanglement by using multiple branches of tree-structured graph convolution layers. Instead of supervising each part respectively, it conducts overall supervision after assembling all the parts. Considering that the parts of MRGAN lack semantic meaning, ? propose EditVAE, which can achieve semantics-aware point cloud generation. Each branch of EditVAE generates not only parts but also additional part offsets and primitives for auxiliary supervision.

In addition, (???) also play important roles in promoting part-aware point cloud generation.

**Assembly-based Shape Generation** Many datasets, such as ShapeNet-Partseg (?) and PartNet (?), provide part-level semantics. Therefore, many works explore shape generation by assembling parts. Specifically, these works can be roughly categorized into three groups: (1) assemble without generation (????). These methods only reconstruct the parts and achieve the diversity of results by assembling different parts. For example, CompoNet (?) synthesizes "unseen" but reasonable point clouds by varying both the parts and their compositions. ? propose a semantic-part-aware embedding space to realize shape composition and decomposition. PartAttention (?) uses a part-wise attention framework to achieve affine transformation of the decoded parts. (2) assemble after generation (????). In contrast to the first category, the parts used for assembling are generated from a Gaussian distribution. These methods mainly focus on designing part offset networks to efficiently assemble parts. For example, G2LGAN (?) uses global and local GANs to supervise the correlation between the parts and the quality of each part respectively. It also adds a Part Refiner to optimize the generated results, such as removing outliers and completing missing regions. PAGENet (?) generates parts using part-level VAEs and designs a Part Assembler to translate parts based on some anchor parts. (3) assemble while generating (???????). This type of method does not assemble the parts after generating all of them, but rather assembles them progressively during generation. For example, 3D-PRNN (?) proposes a generative recurrent neural network that synthesizes multiple plausible shapes step-by-step based on primitives. This progressive process preserves long-range structural coherence. PQ-Net (?) adopts RNN structure and learns 3D shape representations as a sequential part assembly. ShapeAssembly (?) achieves 3D shape structure synthesis by generating domain-specific language programs. The transformation of the statements in these programs enables ShapeAssembly to control the generated results.

## Multimodal Shape Completion

Shapes with missing semantics can lead to a variety of completion results. For example, ? propose MSC-cGAN. Based on pcl2pcl (?), it adds an additional Gaussian distribution during the transformation from partial to complete point cloud features and an encoder to guarantee completion fidelity to the input partial. Different samples on Gaussian distribution correspond to different completion results. ? introduce PVD, a unified probabilistic formulation, to achieve multimodal shape completion by progressively removing noise from the samples. AutoSDF (?) utilizes a transformer-based autoregressive model to generate patch embeddings extracted independently by VQ-VAE (?) step-by-step. The idea of ShapeFormer (?) is similar to AutoSDF. The difference is that AutoSDF embeds the whole 3D space while ShapeFormer introduces a compact 3D representation VQDIF that embeds only the space occupied by 3D shapes, making it more efficient. There are also many other works (????) exploring multimodal shape completion.

## Method

In this section, we first describe the architecture of SGAS according to the proposed four-stage point cloud part editing process, and then give the loss functions of SGAS.

### Segmentation

To achieve the disentanglement between parts which can guarantee fidelity in part editing, SGAS is designed to have multiple branches. Each branch generates a part and requires a Ground Truth part for supervision. In our opinion, parts can be semantic parts, such as a chair back, seat, and base, and can also be local areas of a shape’s surface. The former can directly use some datasets (??) with part semantic labels to obtain Ground Truth parts, while the latter need some unsupervised shape co-segmentation methods (???) to obtain Ground Truth parts. We follow the idea of I-GAN (?), which demonstrates that generating on features is better than directly generating on point clouds. Therefore, using the segmented Ground Truth parts, we pre-train an autoencoder for each semantic part to convert point clouds into features. The encoder is the same as PointNet (?) encoder. The decoder uses a fully connected network. Earth Movers Distance(EMD) (?) is used to supervise the training of these autoencoders. As shown in Figure 2, the trained encoders  $E_{pi}, i = 1...n$  and decoders  $D_{pi}, i = 1...n$  are used to build SGAS. They do not update parameters during SGAS training.

### Generation

The input of SGAS includes not only Gaussian noise but also unedited parts. The purpose is to realize that the style of generated parts matches that of unedited parts, thereby ensuring the quality of the final edited results. We use AdaIN Layer (?) to integrate the unedited parts into the Gaussian distribution. Specifically, a PointNet encoder  $E$  is used to encode the unedited parts to the mean  $\mu$  and standard deviation  $\Sigma$  of a Gaussian distribution. The  $\mu$  and  $\Sigma$  are then applied to the standard Gaussian distribution to obtain a new

Gaussian distribution  $\mathcal{N}(\mu, \Sigma^2)$ . Based on this new distribution, we design several part-level GANs to generate parts. As shown in Figure 2, a Gaussian noise  $z \in \mathbb{R}^{128} \sim \mathcal{N}(\mu, \Sigma^2)$  is transformed by generators  $G_{pi}, i = 1...n$  into part latent features to realize feature disentanglement. The generator uses a 3-layer fully connected network (256, 512, 128). The dimension of the part latent feature is 128. Part features are then sent to discriminators  $F_{pi}, i = 1...n$  to distinguish real parts and generated parts. The discriminator uses a 3-layer fully connected network (256, 512, 1). These part-level discriminators ensure the quality of each part.

### Assembly

Since the branches used for part generation are independent, the generated parts may not be assembled into a reasonable object. To solve this problem, as shown in Figure 2, we add a global discriminator  $F$  in SGAS to supervise all generated parts simultaneously, which realizes feature constraint. The discriminator uses a 3-layer fully connected network (256, 512, 1). Compared to methods (????) that use affine transformation to realize part assembly, our global discriminator can further ensure matching between the generated parts. Since some parts of the target point cloud already exist, we use Part Mask to replace some generated parts. Specifically, we first use pre-trained encoders  $E_{pi}, i = 1...n$  to obtain the part latent features of unedited parts. These part latent features are then used to replace the corresponding generated part features. Finally, the replaced features are sent to discriminator  $F$ .

### Selection

In a real scene, an object does not necessarily contain all semantic parts. For example, a chair without arms and a lamp without a holder. In order to realize this, we perform Part Select on the part features processed after Part Mask. Part Select uses a threshold  $\tau$  to filter the parts that SGAS thinks do not need to be output. It does not need training. Specifically, since some parts might not exist in the Ground Truth point clouds, we set the latent features corresponding to these parts to zero. Therefore, the trained SGAS can automatically determine whether a part needs to be output. It forces the features of parts that do not need to be output as close to zero as possible. In this way, By not decoding the parts whose features are within the threshold  $\tau$ , we realize part selection in the output point clouds. The filter conditions for Part Select are given as:

$$\left| \frac{1}{n} \sum_{i=1}^n G_{pi}(z) \right| \leq \tau \quad (1)$$

where  $|\cdot|$  represents absolute value,  $n$  is the number of parts.

### Loss Functions

We adopt the loss function introduced in Wasserstein GAN (?) with gradient penalty (?). Network  $E, G_{pi}, i = 1...n, F_{pi}, i = 1...n$ , and  $F$  need training. The losses are

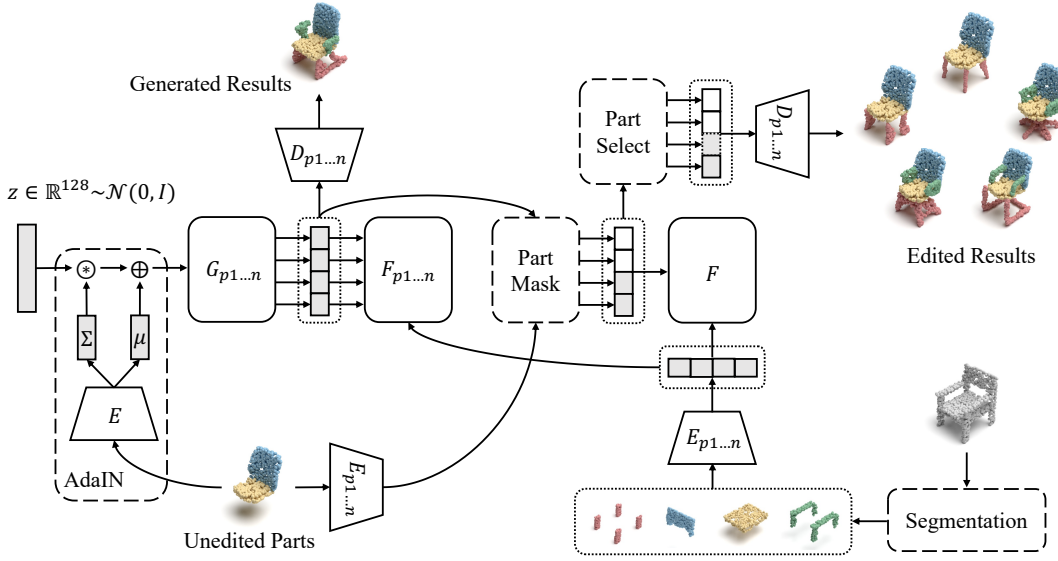


Figure 2: The architecture of SGAS. The inputs are Gaussian noise and unedited parts. The outputs are diverse generated or edited results. SGAS obtains Ground Truth parts through segmentation and uses them to pre-train part-level autoencoders, which convert point clouds into features. By incorporating the style of unedited parts into the Gaussian distribution using an AdaIN layer and performing part-level GAN supervision, SGAS can generate new parts. To constrain each part to form a reasonable object, SGAS applies part masking and uses a global discriminator. Finally, SGAS performs part selection on each part feature, allowing the model to autonomously choose which parts do not need to be output.

given as:

$$\mathcal{L}_G = -\alpha * \frac{1}{n} \sum_{i=1}^n \mathbb{E}_{z \sim Z} [F_{pi}(G_{pi}(z))] - \beta * \mathbb{E}_{z \sim Z} [F(\bigcup_{i=1}^n G_{pi}(z))] \quad (2)$$

$$\mathcal{L}_{F_p} = \frac{1}{n} \sum_{i=1}^n (\mathbb{E}_{z \sim Z} [F_{pi}(G_{pi}(z))] - \mathbb{E}_{x_i \sim R_{xi}} [F_{pi}(x_i)]) + \lambda_{gp} \mathbb{E}_{\hat{x}_i} [(\|\nabla_{\hat{x}_i} F_{pi}(\hat{x}_i)\|_2 - 1)^2] \quad (3)$$

$$\mathcal{L}_F = \mathbb{E}_{z \sim Z, x_i \sim P_{xi}} [F(\bigcup_{i=1}^n M_i G_{pi}(z)) + (1 - M_i) E_{pi}(x_i)] - \mathbb{E}_{x_i \sim R_{xi}} [F(\bigcup_{i=1}^n x_i)] + \lambda_{gp} \mathbb{E}_{\hat{x}_i} [(\|\nabla_{\hat{x}_i} F(\bigcup_{i=1}^n \hat{x}_i)\|_2 - 1)^2] \quad (4)$$

where  $\mathcal{L}_G$ ,  $\mathcal{L}_{F_p}$ , and  $\mathcal{L}_F$  represent the loss functions of the part-level generator, the part-level discriminator, and the global discriminator respectively.  $\alpha$  and  $\beta$  are hyperparameters that control the proportion of the part-level to the global GAN.  $N$  is the number of parts.  $x_i, i = 1 \dots n$  are parts. The formulas after  $\lambda_{gp}$  are the gradient penalty terms proposed by ?.  $Z = \mathcal{N}(\mu, \Sigma^2)$ , where  $\mu$  and  $\Sigma$  are from encoder  $E$ .  $M$  is determined by the unedited parts.

## Experiments

### Datasets and Implementation Details

We evaluate SGAS on PartNet (?) dataset. By merging fine-grained semantic labels and removing some special objects, we create a new dataset called PartNet.v0.Merged for point cloud part editing. Following previous works (??), we perform unsupervised part-aware point cloud generation on ShapeNet-Partseg (?) dataset. We do not use semantic labels in the ShapeNet-Partseg dataset.

Table 1: Diversity part editing performance. MMD and TMD measure the quality and diversity respectively.

|       | Model    | Chair       | Lamp        | Table       | Average     |
|-------|----------|-------------|-------------|-------------|-------------|
| MMD ↓ | MSC-cGAN | 1.62        | 3.41        | 1.39        | 2.14        |
|       | SGAS     | <b>1.33</b> | <b>2.26</b> | <b>1.06</b> | <b>1.55</b> |
| TMD ↑ | MSC-cGAN | <b>5.45</b> | 3.94        | 5.14        | 4.84        |
|       | SGAS     | 4.36        | <b>4.48</b> | <b>8.04</b> | <b>5.63</b> |

Adam optimizers are used for SGAS with a learning rate of  $\alpha = 0.0005$ , coefficients  $\beta_1 = 0.5$  and  $\beta_2 = 0.99$ . All the experiments are performed on a single NVIDIA TITAN Xp for 2000 epochs with a batch size of 200. In loss functions,  $\alpha$  and  $\beta$  are set to 1 and 1.  $\lambda_{gp}$  is set to 10. The threshold in Part Select is set to 0.5. We update the discriminator 5 times for each update of the generator. Each shape has 2048 points while each part has  $\lfloor \frac{2048}{n} \rfloor$  points.  $n$  is the number of parts. During training, the input unedited parts for SGAS are obtained by randomly removing 1 to  $n - 1$  parts of objects

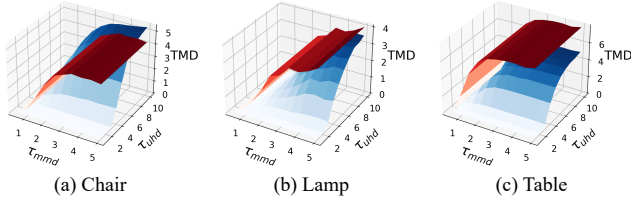


Figure 3: Performance on the new metric Total Mutual Difference Surface (TMDS). Red represents SGAS; blue represents MSC-cGAN. The smaller the thresholds of MMD and UHD are, the more referential the calculated TMD is.

in PartNet.v0.Merged dataset.

## Point Cloud Part Editing

Diversity part editing is a commonly used operation in point cloud part editing that involves generating some parts multiple times to obtain various results. Previous part editing methods (???) lack related evaluation metrics. Since diversity part editing has some intersects with multimodal shape completion, here we use the metrics MMD, TMD, and UHD adopted by ? to measure the quality, diversity, and fidelity of the edited results respectively. Our SGAS disentangles each part, so the input unedited parts can remain unchanged. Therefore, we only compare MMD and TMD. Considering previous part editing methods can only perform part editing on generated objects, which makes it impossible to obtain Gaussian noise corresponding to existing objects for editing. Hence, we use the representative multimodal shape completion method MSC-cGAN (?) as the baseline for the this study. As shown in Table 1, SGAS achieves excellent results in three representative categories.

Table 2: Ablation results for the hyperparameters  $\alpha$  and  $\beta$ . The set of  $\alpha : \beta$  can be determined by requirement.

| $\alpha : \beta$ | 1:10 | 1:2  | 1:1         | 2:1  | 5:1  | 10:1        |
|------------------|------|------|-------------|------|------|-------------|
| MMD ↓            | 1.47 | 1.39 | <b>1.33</b> | 1.35 | 1.36 | 1.40        |
| TMD ↑            | 1.89 | 2.65 | 4.36        | 4.75 | 5.34 | <b>7.68</b> |

During experiments, we find that the diversity metric TMD only measures the difference between edited results without considering fidelity and quality, which means two incorrect situations that can also result in high TMD: (a) the change of input unedited parts (corresponds to large UHD); (b) the edited parts with large differences but poor quality (corresponds to large MMD). Therefore, we further propose a new metric TMDS (TMD Surface) to solve the problems.

Each point value on the surface is calculated as:

$$\text{TMDS}(\tau_{\text{uhd}}, \tau_{\text{mmd}}) = \text{mean}_{p \in \mathbb{P}} \begin{cases} \text{TMD}(s_1, \dots, s_k), & \text{if } \exists s_i, i = 1 \dots k, \\ & \text{UHD}(p, s_i) \leq \tau_{\text{uhd}} \\ & \text{MMD}(s_i, \mathbb{D}) \leq \tau_{\text{mmd}} \\ 0, & \text{otherwise} \end{cases}$$

$$\text{TMD}(s_1, \dots, s_k) = \sum_{i=1}^k \frac{1}{k-1} \sum_{j \neq i, j=1}^k \text{CD}(s_i, s_j) \quad (5)$$

where  $p$  is input unedited parts,  $s_i, i = 1 \dots k$  are  $K$  (e.g. 10) edited results.  $\mathbb{P}$  is the unedited parts test dataset and  $\mathbb{D}$  is original test dataset.  $\text{CD}$  is Chamfer Distance (?).  $\tau_{\text{uhd}}$  and  $\tau_{\text{mmd}}$  are thresholds of UHD and MMD respectively. TMDS requires that each point value on the surface is calculated when  $K$  edited results are guaranteed to satisfy the corresponding UHD and MMD thresholds. The smaller the thresholds of MMD and UHD are, the more referential the calculated TMD is. Therefore, as shown in Figure 3, we can find that the editing diversity of SGAS is better than that of MSC-cGAN in the Chair category.

We use SGAS to perform various part editing operations on PartNet.v0.Merged dataset. Figure 4(a) is the visualized comparison of the diversity edited results. We also perform our SGAS on three new categories: Display, Knife, and Mug. It can be found that SGAS can not only keep the input unchanged but also have higher editing diversity and quality. Figure 4(b) is a multi-round editing workflow achieved by SGAS. It demonstrates SGAS’s ability to re-edit unsatisfactory edited results. Through three rounds of re-editing, a chair with right-angle arms and wheels is edited into a chair with circular arms and a circular base. Figure 4(c) shows some interpolation part editings. If the chairs in the left and right box are generated by Gaussian noise  $z_s$  and  $z_t$  respectively, the chairs between boxes are generated by Gaussian noise  $z = (1 - \alpha)z_s + \alpha z_t$ .  $\alpha$  increases from 0 to 1. From top to bottom, each row represents an interpolation of one, two, and three parts. We can clearly observe the gradual change process of the parts. Figure 4(d) demonstrates a style mixing part editing realized by SGAS. We first select four chairs with different styles. Then we edit different parts (colored) in different chairs. Finally, these edited parts are assembled to obtain chairs with styles from different chairs. This editing operation helps to create more diverse results.

As the hyperparameters  $\alpha$  and  $\beta$  represent diversity and quality respectively, and have a significant impact on the edited results. Hence, we conduct an ablation study on them. The results are presented in Table 2. It is observed that as  $\alpha : \beta$  increase, the TMD also increase. However, the MMD, which measures the quality of the edited results, initially improves but then worsens. Therefore, we finally choose  $\alpha : \beta = 1 : 1$  to realize part editing. However, if diversity is more important for the edited results, a higher  $\alpha : \beta$  can also be used. To demonstrate SGAS’s ability to automatically select parts, we visualize the latent features of the edited parts. As shown in Figure 5, it can be found that the latent feature of the chair arm is near zero, which means

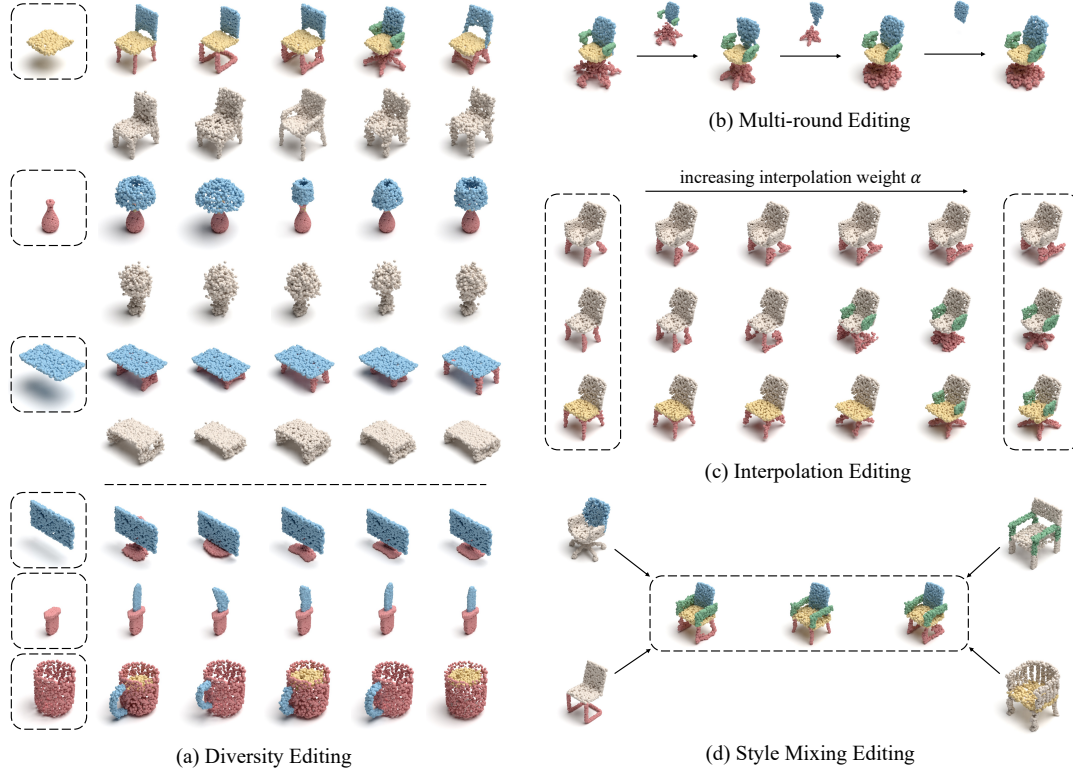


Figure 4: Various part editing operations. (a) Diversity editing comparison. The unedited parts are boxed, followed by five different edited results. The results of MSC-cGAN are uncolored while ours are colored by parts. (b) Re-editing of unsatisfactory edited results. The parts above the arrow are edited in each round. (c) Continuous transformation of selected parts (colored) through interpolation of two input Gaussian noises. Each row represents an interpolation of different number of parts. (d) Style Mixing of different parts in different objects. The mixed results are boxed.

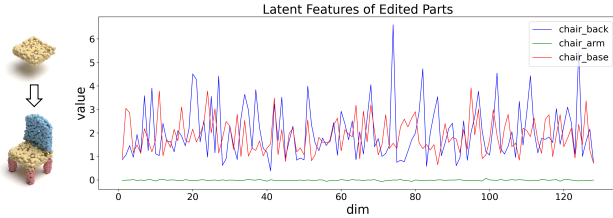


Figure 5: Edited parts with their corresponding latent features (128 dim). The chair without outputting arms has its corresponding latent feature of the chair arm near zero.

SGAS believes that the newly generated chair arm is inappropriate. Therefore, the Part Select module in SGAS will filter this latent feature to prevent the chair arm from being output. To further prove the generalization ability of SGAS, we train SGAS on PartNet.v0.Merged dataset and test it on ScanNet (?) dataset. The results can be found in Figure 6. For parts with high missing rates, we will regenerate them, such as the chair back in the 1st row, and the newly generated parts are compatible with the existing incomplete parts. For parts with low missing rates, we will keep them directly. It can be found that even on unseen objects, SGAS’s diversity

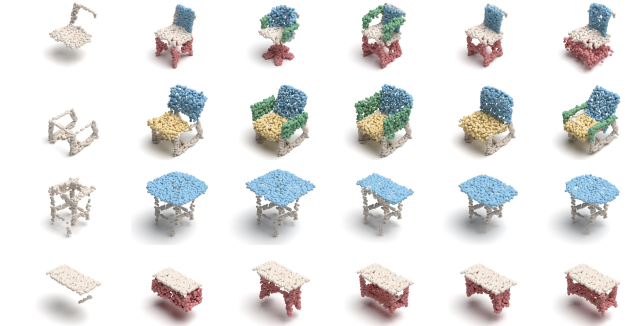


Figure 6: Performance on ScanNet. The leftmost column is incomplete input, followed by five diversity editing results.

editing results are still good.

### Unsupervised Part-aware Point Cloud Generation

By pruning SGAS, including removing unedited parts input, AdaIN Layer, Part Mask, and Part Select, SGAS can be applied to realize unsupervised part-aware point cloud generation. It includes two steps: (a) modifying an unsupervised shape co-segmentation method AXform (?) to get

Table 3: Generative performance. The optimal and suboptimal results are highlighted in red and blue respectively.  $M$  and  $N$  represent the number of parts.

| Category | Model         | JSD ↓        | MMD ↓         |              | COV %, ↑  |           |
|----------|---------------|--------------|---------------|--------------|-----------|-----------|
|          |               |              | CD            | EMD          | CD        | EMD       |
| Chair    | TreeGAN       | 0.119        | <b>0.0016</b> | 0.101        | 58        | 30        |
|          | MRGAN         | 0.246        | 0.0021        | 0.166        | <b>67</b> | 23        |
|          | EditVAE (M=7) | 0.063        | <b>0.0014</b> | <b>0.082</b> | 46        | 32        |
|          | EditVAE (M=3) | <b>0.031</b> | 0.0017        | 0.101        | 45        | <b>39</b> |
|          | SGAS (N=7)    | <b>0.047</b> | 0.0020        | <b>0.076</b> | <b>60</b> | <b>58</b> |
| Airplane | TreeGAN       | 0.097        | 0.0004        | 0.068        | 61        | 20        |
|          | MRGAN         | 0.243        | 0.0006        | 0.114        | <b>75</b> | 21        |
|          | EditVAE (M=6) | <b>0.043</b> | <b>0.0004</b> | <b>0.024</b> | 39        | <b>30</b> |
|          | EditVAE (M=3) | 0.044        | 0.0005        | 0.067        | 23        | 17        |
|          | SGAS (N=6)    | <b>0.036</b> | <b>0.0004</b> | <b>0.039</b> | <b>61</b> | <b>58</b> |
| Table    | TreeGAN       | 0.077        | 0.0018        | 0.082        | <b>71</b> | <b>48</b> |
|          | MRGAN         | 0.287        | 0.0020        | 0.155        | <b>78</b> | 31        |
|          | EditVAE (M=5) | 0.081        | <b>0.0016</b> | <b>0.071</b> | 42        | 27        |
|          | EditVAE (M=3) | <b>0.042</b> | <b>0.0017</b> | 0.130        | 39        | 30        |
|          | SGAS (N=5)    | <b>0.057</b> | 0.0020        | <b>0.069</b> | 65        | <b>65</b> |

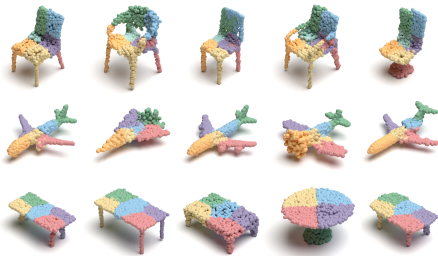


Figure 7: Point clouds generated by SGAS, colored by parts.

Ground Truth part datasets; (b) training SGAS on these part datasets. Specifically, we first modify the multi-branch AX-form to output one structure point per branch. Second, we use these structure points to co-segment the Ground Truth point clouds into  $n$  part datasets. Third, the segmented parts are pre-encoded into latent features. Finally, these latent features are used to supervise the training of SGAS. However, we found that there might be some large gaps between parts during generation. Therefore, to achieve seamless generation, during unsupervised part segmentation, we further expand the number of points per part from  $\lfloor \frac{2048}{n} \rfloor$  to  $(1 + \gamma) \lfloor \frac{2048}{n} \rfloor$ . Here  $\gamma = 0.1$ . In the final output, we down-sample the point cloud to 2048 points.

The quantitative results are shown in Table 3. The metrics are proposed by ?, and the results of previous methods are obtained from EditVAE (?). MMD and COV represent the quality and diversity of the generated results respectively. It can be found that SGAS achieves excellent results overall, with the most number of top two metrics. Especially on COV-EMD, which represents diversity, SGAS has a significant improvement. Figure 7 gives visualized results of the generated point clouds. Different colors correspond to different parts. It intuitively illustrates the diversity and quality of the results generated by SGAS. We also conduct an ablation study on the number of parts in the Airplane category.

Table 4: Ablation results for the number of parts. The optimal and suboptimal results are highlighted in red and blue respectively.  $n = 6$  is a suitable number of parts.

| #Parts (n) | JSD ↓        | MMD ↓         |              | COV %, ↑  |           |
|------------|--------------|---------------|--------------|-----------|-----------|
|            |              | CD            | EMD          | CD        | EMD       |
| 2          | 0.042        | 0.0004        | 0.045        | <b>61</b> | 47        |
| 3          | 0.039        | <b>0.0004</b> | 0.043        | 60        | 52        |
| 5          | 0.040        | 0.0005        | 0.042        | 60        | 50        |
| 6          | <b>0.036</b> | <b>0.0004</b> | <b>0.039</b> | <b>61</b> | <b>58</b> |
| 8          | 0.040        | 0.0005        | 0.044        | 60        | 46        |
| 13         | <b>0.035</b> | 0.0005        | <b>0.040</b> | 60        | <b>53</b> |

As shown in Table 4, more or fewer parts are not necessarily beneficial to the results. Therefore, we chose the number of parts  $N = 6$ .

## Conclusion

Previous methods do not disentangle each part completely or lack constraints between parts, which leads to poor diversity, fidelity, and quality when performing part editing. In this work, to solve these problems, we first propose a novel four-stage point cloud part editing process. Then based on this process and two new strategies: feature disentanglement and constraint, we propose a part editing model SGAS. It can realize various part editing operations. By introducing metrics from multimodal completion and proposing a new metric TMDS, we establish quantitative evaluations for diversity part editing. In addition, SGAS can be pruned to realize unsupervised part-aware point cloud generation. Experiments show that it performs better than previous methods.

**Limitation** Since we do not design part offset networks for the generated parts but instead utilize the relatively fixed spatial positions of each generated part to ensure good assembly, SGAS can only achieve part editing for objects with relatively consistent prototypes. For example, SGAS cannot handle part editing for both ceiling lamps and table lamps simultaneously as the spatial order of the parts is opposite. In addition, we also find that the performance of SGAS is limited by the pre-trained autoencoders. The embedded features are better when the parts are normalized. Therefore, it will be beneficial to first generate normalized parts and then design part offset networks to align them in the future.

## Acknowledgments

This work was supported by National Natural Science Fund of China (62176064). Cheng Jin is the corresponding author.

Laborum doloremque veniam enim aliquid reprehenderit consequatur fuga, nostrum blanditiis voluptatibus illum nemo non earum ad magni?Quam quia ipsum cum recusandae velit nihil molestiae atque, maxime quos ducimus labore laboriosam, quae placeat reprehenderit, accusamus dolore consequuntur natus suscipit asperiores fugiat reprehenderit corporis eum quam?Est suscipit asperiores laboriosam aperiam quisquam ea dignissimos pariatur totam assumenda voluptate, assumenda at magnam ipsum pariatur?Laborum molestiae amet veniam dicta magnam, suscipit dolor libero

nemo omnis consequuntur itaque facere similique, esse soluta a aliquid, totam illum omnis ea dolor, deserunt accusantium eos similique?